

Listing 8-6: The `Student` class with property decorators

The `__init__()` method has a little trick to it, so let's examine the other methods first. Notice we have two methods with the name `grade()`. Preceding the definition of the first `grade()` method, we add an `@property` decorator ❷. This defines the name `grade` as a property of any object created from this class. The first method ❸ is a getter that just returns the value of the current grade, kept in the private `self.__grade` instance variable, but could include any code that might be needed to calculate a value and return it.

Preceding the second `grade()` method is an `@grade.setter` decorator ❹. This second method ❺ accepts a new value as a parameter, does a number of checks to ensure that value is valid, then sets the new value into `self.__grade`.

The `__init__()` method first stores the student's name in an instance variable. The next line ❶ seems straightforward but is a little unusual. As we have seen, we typically store the values of parameters into instance variables. Therefore, we might be tempted to write this line as:

```
self.__grade = startingGrade
```

But instead, we are storing the starting grade into the property `grade`. Since `grade` is a property, Python translates this assignment statement into a call to the setter method ❺, which has the advantage of validating the input before storing the value in the instance variable `self.__grade`.

[Listing 8-7](#) provides some test code that uses the `Student` class.

File: PropertyDecorator/Main_Property.py

```
# Main Student property example
❶ oStudent1= Student('Joe Schmoe')
   oStudent2= Student ('Jane Smith')

# Get the students' grades using the 'grade' property and print
❷ print(oStudent1.grade)
   print(oStudent2.grade)
   print()

# Set new values using the 'grade' property
```
